

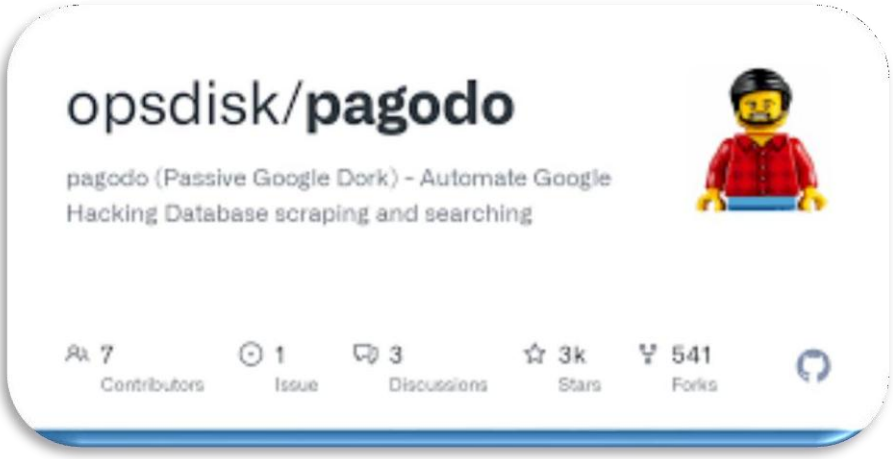
Abdul Wahid.

CTI Lab Report

Category: Automated Passive Recon — GHDB & Pagodo Dork Automation

Section 1 — Environment Setup & Tool Installation

The first step in the lab was setting up the required environment on a Windows 10 system. Since Git was not pre-installed, it was installed via the Windows Package Manager (winget) before cloning the Pagodo repository from GitHub.



— Git Installation

Git was absent from the system PATH initially. The following winget command was used to install the latest stable version:

```
C:\Users\micro> winget install --id Git.Git -e --source winget

Found Git [Git.Git] Version 2.53.0
Downloading Git-2.53.0-64-bit.exe  61.5 MB / 61.5 MB
Successfully installed

C:\Users\micro> git --version
git version 2.53.0.windows.1
```

— Cloning the Pagodo Repository

With Git installed, the Pagodo tool was cloned directly from the official GitHub repository:

```
C:\Users\micro>git clone https://github.com/opsdisk/pagodo.git
Cloning into 'pagodo'...
remote: Enumerating objects: 575, done.
remote: Counting objects: 100% (181/181), done.
remote: Compressing objects: 100% (113/113), done.
remote: Total 575 (delta 85), reused 68 (delta 68), pack-reused 394 (from 4)
Receiving objects: 100% (575/575), 1.89 MiB | 109.00 KiB/s, done.
Resolving deltas: 100% (320/320), done.
```

— Python & Dependencies

Python 3.14.3 was confirmed available via the 'py' alias. pip and setuptools were upgraded, and all project dependencies were installed from requirements.txt:

Navigate to pagodo:

```
C:\Users\micro>cd pagodo
```

Version Checkup:

```
C:\Users\micro\pagodo>py --version
Python 3.14.3
```

Upgrading setup tools:

```
C:\Users\micro\pagodo>py -m pip install --upgrade pip setuptools
```

Installing dependencies:

```
C:\Users\micro\pagodo>py -m pip install -r requirements.txt
Collecting beautifulsoup4==4.13.5 (from -r requirements.txt (line 1))
  Downloading beautifulsoup4-4.13.5-py3-none-any.whl.metadata (3.8 kB)
Collecting requests==2.32.5 (from -r requirements.txt (line 2))
  Downloading requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Collecting yagooglesearch==1.10.0 (from -r requirements.txt (line 3))
  Downloading yagooglesearch-1.10.0-py3-none-any.whl.metadata (13 kB)
Collecting soupsieve>1.2 (from beautifulsoup4==4.13.5->-r requirements.txt (line 1))
  Downloading soupsieve-2.8.3-py3-none-any.whl.metadata (4.6 kB)
Collecting typing-extensions>=4.0.0 (from beautifulsoup4==4.13.5->-r requirements.txt (line 1))
  Using cached typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Collecting charset_normalizer<4,>=2 (from requests==2.32.5->-r requirements.txt (line 2))
  Downloading charset_normalizer-3.4.4-cp314-cp314-win_amd64.whl.metadata (38 kB)
Collecting idna<4,>=2.5 (from requests==2.32.5->-r requirements.txt (line 2))
  Downloading idna-3.11-py3-none-any.whl.metadata (8.4 kB)
```

Step / Action	Status / Result
Git Installation	☑ Version 2.53.0 Installed
Pagodo Repository Cloned	☑ 575 objects received
Python Version	☑ Python 3.14.3 Confirmed
pip Upgraded	☑ pip 26.0.1 Installed
Requirements Installed	☑ All 10 packages successful

Section 2 — Pagodo Configuration & GHDB Dork Scraping

After confirming the tool was functional, Pagodo's help menu was reviewed to understand all available flags and options. This step is critical before running any scan to ensure proper configuration.

— Reviewing Pagodo Options

The -h flag was used to display all available Pagodo parameters:

```
C:\Users\micro\pagodo>py pagodo.py -h
usage: pagodo.py [-h] -g GOOGLE_DORKS_FILE [-d DOMAIN] [-i MINIMUM_DELAY_BETWEEN_DORK_SEARCHES_IN_SECONDS]
                 [-x MAXIMUM_DELAY_BETWEEN_DORK_SEARCHES_IN_SECONDS] [-l] [-m MAX_SEARCH_RESULT_URLS_TO_RETURN_PER_DORK]
                 [-p PROXIES] [-o [JSON_FILE]] [-s [URL_FILE]] [-v VERBOSITY] [-z SPECIFIC_LOG_FILE_NAME]

pagodo - Passive Google Dork v2.7.0

options:
  -h, --help            show this help message and exit
  -g, --google-dorks-file GOOGLE_DORKS_FILE
                        File containing Google dorks, 1 per line.
  -d, --domain DOMAIN   Domain to scope the Google dork searches. Not required.
  -i, --minimum-delay-between-dork-searches MINIMUM_DELAY_BETWEEN_DORK_SEARCHES_IN_SECONDS
                        Minimum delay (in seconds) between a Google dork search. Default: 37
  -x, --maximum-delay-between-dork-searches MAXIMUM_DELAY_BETWEEN_DORK_SEARCHES_IN_SECONDS
                        Maximum delay (in seconds) between a Google dork search. Default: 60
  -l, --disable-ssl-verification
                        Disable SSL/TLS validation. Sometimes required if using an HTTPS proxy with self-signed certificates.
  -m, --max-search-urls-to-return-per-dork MAX_SEARCH_RESULT_URLS_TO_RETURN_PER_DORK
                        Maximum results to return per dork. Default 100.
  -p, --proxies PROXIES
                        Comma separated string of proxies to round-robin through. Example:
                        https://myproxy:8080,socks5h://127.0.0.1:9050,socks5h://127.0.0.1:9051 - The proxy scheme must conform
                        per the Python requests library: https://docs.python-requests.org/en/master/user/advanced/#proxies Also
                        see https://github.com/opsdisk/yagooglesearch for more information.
  -o, --json-results-file [JSON_FILE]
                        Save URL dork data to a JSON file.  Contains more information than .txt version
                        no -o = Do not save dork data to a JSON file
                        -o = Save dork data to pagodo_results_<TIMESTAMP>.json
                        -o JSON_FILE = Save dork data to JSON_FILE
  -s, --text-results-file [URL_FILE]
                        Save URL dork data to a text file.
                        no -s = Do not save dork data to a file
                        -s = Save dork data to pagodo_results_<TIMESTAMP>.txt
                        -s URL_FILE = Save dork data to URL_FILE
  -v, --verbosity VERBOSITY
                        Verbosity level (0=NOTSET, 1=CRITICAL, 2=ERROR, 3=WARNING, 4=INFO, 5=DEBUG). Default: 4
  -z, --log SPECIFIC_LOG_FILE_NAME
                        Save log data to a specific log filename, otherwise the default is pagodo.py.log
```

— Downloading All GHDB Dorks

The ghdb_scraper.py script was executed to pull all available dorks from the Exploit-DB Google Hacking Database. This created both a JSON and TXT version of the complete dork library locally:

```
C:\Users\micro\pagodo>py ghdb_scraper.py -j -s
[+] Requesting URL: https://www.exploit-db.com/google-hacking-database
[*] Writing all dorks to JSON file: all_google_dorks.json
[*] Writing all dorks to txt file: dorks/all_google_dorks.txt
[*] Total Google dorks retrieved: 7944
```

Section 3 — Script 1: GHDB Dork Filtering

Running all 7,944 dorks against a target would take several hours due to mandatory delays needed to avoid Google IP bans. A custom Python filtering script was developed to extract only the most relevant dorks from the GHDB JSON file based on target-specific keywords and high-value categories.

— The Problem

7,944 dorks × an average 50-second delay = approximately 110 hours of scanning time. This is completely impractical for a real engagement. The solution was to intelligently filter dorks before scanning.

— Filter Script Design (filter_dorks.py)

The script loads the full GHDB JSON file, extracts the actual dork string from each HTML anchor tag, then filters dorks by matching against two criteria: high-value GHDB categories and specific sensitive keywords.

```
import json
import re

# Load all dorks
with open('dorks/all_google_dorks.json', 'r', encoding='utf-8') as f:
    data = json.load(f)

# ---- CONFIG ----
KEYWORDS = ['twitter', 'admin', 'password', 'config',
            'backup', 'env', 'sql', 'login', 'secret', 'key']

CATEGORIES = [
    'Footholds',
    'Files Containing Juicy Info',
    'Vulnerable Files',
    'Vulnerable Servers',
    'Files Containing Passwords',
    'Sensitive Directories'
]

OUTPUT_FILE = 'filtered_dorks.txt'

# ---- Extract & Filter ----
filtered = []

for dork in data:
    raw = dork.get('url_title', '')
    match = re.search(r'>(.+?)</a>', raw)
    if not match:
        continue

    actual_dork = match.group(1).strip()
    category = dork.get('cat_id', ['', ''])[1]

    category_match = any(cat.lower() in category.lower() for cat in CATEGORIES)
    keyword_match = any(kw.lower() in actual_dork.lower() for kw in KEYWORDS)
```

```
    if category_match or keyword_match:
        filtered.append(actual_dork)

filtered = list(set(filtered))

with open(OUTPUT_FILE, 'w', encoding='utf-8') as f:
    for dork in filtered:
        f.write(dork + '\n')

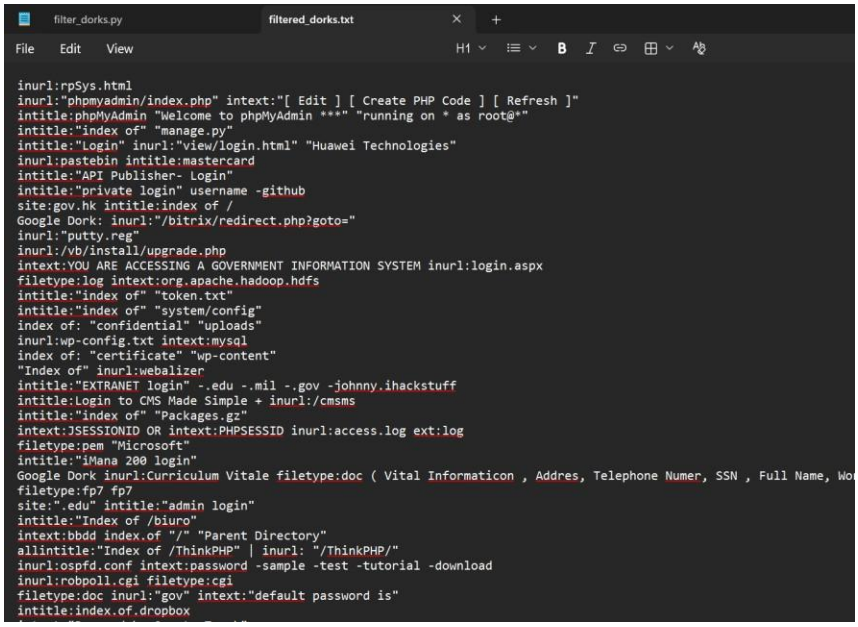
print(f"[+] Total dorks from GHDB: {len(data)}")
print(f"[+] Filtered relevant dorks: {len(filtered)}")
print(f"[+] Saved to: {OUTPUT_FILE}")
print(f"\n[Preview - First 10 Dorks:]")
for d in filtered[:10]:
    print(f"  {d}")
```

— Filter Script Output

```
C:\Users\micro\pagodo>py filter_dorks.py
[+] Total dorks from GHDB: 7944
[+] Filtered relevant dorks: 4181
[+] Saved to: filtered_dorks.txt

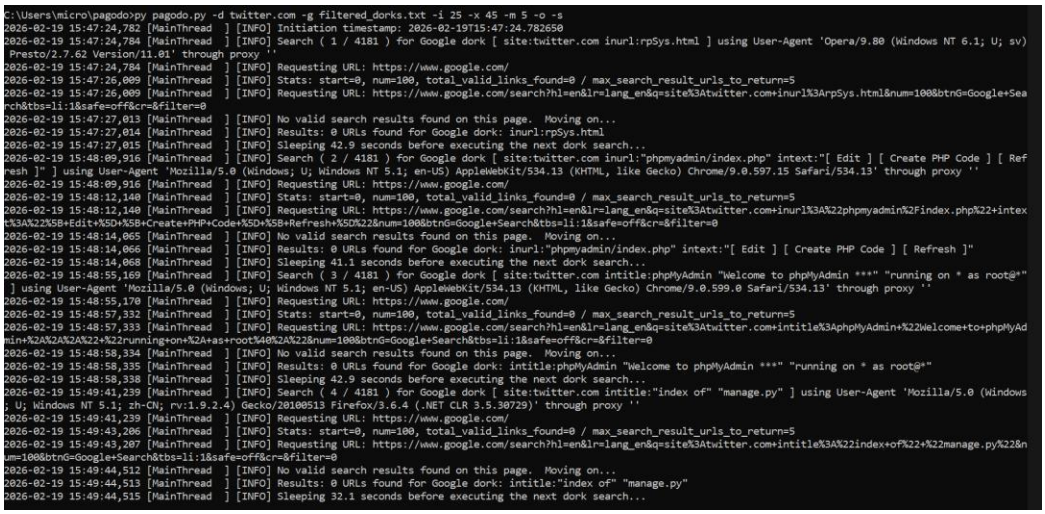
[Preview - First 10 Dorks:]
  inurl:rpSys.html
  inurl:"phpmyadmin/index.php" intext:"[ Edit ] [ Create PHP Code ] [ Refresh ]"
  intitle:phpMyAdmin "Welcome to phpMyAdmin ***" "running on * as root@*"
  intitle:"index of" "manage.py"
  intitle:"Login" inurl:"view/login.html" "Huawei Technologies"
  inurl:pastebin intitle:mastercard
  intitle:"API Publisher- Login"
  intitle:"private login" username -github
  site:gov.hk intitle:index of /
  Google Dork: inurl:"/bitrix/redirect.php?goto="
```

Filtered txt file created:



Step / Action	Status / Result
Total GHDB Dorks	**7,944**
Filtered Relevant Dorks	**4,181 (52.6% reduction)**
Output File	**filtered_dorks.txt ☑**
Time Saved	**~50% reduction in scan time**

Targeting twitter from the filtered dorks (test lab environment)



```
C:\Users\micro\pagodo>py pagodo.py -d twitter.com -g filtered_dorks.txt -i 25 -x 45 -m 5 -o -s
2026-02-19 15:47:24,782 [MainThread  ] [INFO] Initiation timestamp: 2026-02-19T15:47:24.782650
2026-02-19 15:47:24,784 [MainThread  ] [INFO] Search ( 1 / 4181 ) for Google dork [ site:twitter.com inurl:rpSys.html ] using User-Agent 'Opera/9.80 (Windows NT 6.1; U; sv)
 Presto/2.7.62 Version/11.01' through proxy ''
2026-02-19 15:47:24,784 [MainThread  ] [INFO] Requesting URL: https://www.google.com/
2026-02-19 15:47:26,009 [MainThread  ] [INFO] Stats: start=0, num=100, total_valid_links_found=0 / max_search_result_urls_to_return=5
2026-02-19 15:47:26,009 [MainThread  ] [INFO] Requesting URL: https://www.google.com/search?hl=en&lr=lang_en&q=site%3Atwitter.com+inurl%3ArpSys.html&num=100&btnG=Google+Sea
rch&tbs=li:1&safe=off&cr=&filter=0
2026-02-19 15:47:27,013 [MainThread  ] [INFO] No valid search results found on this page.  Moving on...
2026-02-19 15:47:27,014 [MainThread  ] [INFO] Results: 0 URLs found for Google dork: inurl:rpSys.html
2026-02-19 15:47:27,015 [MainThread  ] [INFO] Sleeping 42.9 seconds before executing the next dork search...
2026-02-19 15:48:09,916 [MainThread  ] [INFO] Search ( 2 / 4181 ) for Google dork [ site:twitter.com inurl:"phpmyadmin/index.php" intext:"[ Edit ] [ Create PHP Code ] [ Ref
resh ]" ] using User-Agent 'Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US) AppleWebKit/534.13 (KHTML, like Gecko) Chrome/9.0.597.15 Safari/534.13' through proxy ''
2026-02-19 15:48:09,916 [MainThread  ] [INFO] Requesting URL: https://www.google.com/
2026-02-19 15:48:12,140 [MainThread  ] [INFO] Stats: start=0, num=100, total_valid_links_found=0 / max_search_result_urls_to_return=5
2026-02-19 15:48:12,140 [MainThread  ] [INFO] Requesting URL: https://www.google.com/search?hl=en&lr=lang_en&q=site%3Atwitter.com+inurl%3A%22phpmyadmin%2Findex.php%22+intex
t%3A%22%5B+Edit+%5D+%5B+Create+PHP+Code+%5D+%5B+Refresh+%5D%22&num=100&btnG=Google+Search&tbs=li:1&safe=off&cr=&filter=0
2026-02-19 15:48:14,065 [MainThread  ] [INFO] No valid search results found on this page.  Moving on...
2026-02-19 15:48:14,066 [MainThread  ] [INFO] Results: 0 URLs found for Google dork: inurl:"phpmyadmin/index.php" intext:"[ Edit ] [ Create PHP Code ] [ Refresh ]"
2026-02-19 15:48:14,068 [MainThread  ] [INFO] Sleeping 41.1 seconds before executing the next dork search...
2026-02-19 15:48:55,169 [MainThread  ] [INFO] Search ( 3 / 4181 ) for Google dork [ site:twitter.com intitle:phpMyAdmin "Welcome to phpMyAdmin ***" "running on * as root@*"
 ] using User-Agent 'Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US) AppleWebKit/534.13 (KHTML, like Gecko) Chrome/9.0.599.0 Safari/534.13' through proxy ''
2026-02-19 15:48:55,170 [MainThread  ] [INFO] Requesting URL: https://www.google.com/
2026-02-19 15:48:57,332 [MainThread  ] [INFO] Stats: start=0, num=100, total_valid_links_found=0 / max_search_result_urls_to_return=5
2026-02-19 15:48:57,333 [MainThread  ] [INFO] Requesting URL: https://www.google.com/search?hl=en&lr=lang_en&q=site%3Atwitter.com+intitle%3AphpMyAdmin+%22Welcome+to+phpMyAd
min+%2A%2A%2A%22+%22running+on+%2A+as+root%40%2A%22&num=100&btnG=Google+Search&tbs=li:1&safe=off&cr=&filter=0
2026-02-19 15:48:58,334 [MainThread  ] [INFO] No valid search results found on this page.  Moving on...
2026-02-19 15:48:58,335 [MainThread  ] [INFO] Results: 0 URLs found for Google dork: intitle:phpMyAdmin "Welcome to phpMyAdmin ***" "running on * as root@*"
2026-02-19 15:48:58,338 [MainThread  ] [INFO] Sleeping 42.9 seconds before executing the next dork search...
2026-02-19 15:49:41,239 [MainThread  ] [INFO] Search ( 4 / 4181 ) for Google dork [ site:twitter.com intitle:"index of" "manage.py" ] using User-Agent 'Mozilla/5.0 (Windows
; U; Windows NT 5.1; zh-CN; rv:1.9.2.4) Gecko/20100513 Firefox/3.6.4 (.NET CLR 3.5.30729)' through proxy ''
2026-02-19 15:49:41,239 [MainThread  ] [INFO] Requesting URL: https://www.google.com/
2026-02-19 15:49:43,206 [MainThread  ] [INFO] Stats: start=0, num=100, total_valid_links_found=0 / max_search_result_urls_to_return=5
2026-02-19 15:49:43,207 [MainThread  ] [INFO] Requesting URL: https://www.google.com/search?hl=en&lr=lang_en&q=site%3Atwitter.com+intitle%3A%22index+of%22+%22manage.py%22&n
um=100&btnG=Google+Search&tbs=li:1&safe=off&cr=&filter=0
2026-02-19 15:49:44,512 [MainThread  ] [INFO] No valid search results found on this page.  Moving on...
2026-02-19 15:49:44,513 [MainThread  ] [INFO] Results: 0 URLs found for Google dork: intitle:"index of" "manage.py"
2026-02-19 15:49:44,515 [MainThread  ] [INFO] Sleeping 32.1 seconds before executing the next dork search...
```

Section 4 — Script 2: Smart Priority-Based Targeting

The second script (smart_dork.py) took filtering one step further by implementing a three-tier priority system. Instead of treating all 4,181 filtered dorks equally, this script ranked dorks by severity and relevance, automatically prefixed them with the target domain, and then launched Pagodo directly — creating a fully automated end-to-end pipeline.

— Priority System Design

Dorks were ranked into three tiers based on the combination of GHDB category severity and the presence of high-value security keywords:

Step / Action	Status / Result
High Priority	**High-value category AND sensitive keyword match**
Medium Priority	**High-value category OR sensitive keyword match**
Low Priority	**Medium-value category match only**

— Auto Target Integration

The script automatically prepended 'site:TARGET' to each dork that didn't already contain a site: operator, ensuring every dork was precisely scoped to the intended target domain. Only the top 100 highest-priority dorks were selected to keep scan time manageable.

```
import json
import re
import subprocess
import sys

# ==========================================
#           CONFIGURATION
# ==========================================
TARGET = "twitter.com"          # Change this to any target
OUTPUT_FILE = "smart_dorks.txt"
MAX_DORKS = 100                 # Keep it small for speed

# High priority categories from GHDB
HIGH_PRIORITY_CATEGORIES = [
    'Footholds',
    'Files Containing Passwords',
    'Vulnerable Files',
    'Vulnerable Servers',
    'Files Containing Usernames',
    'Sensitive Directories',
]

MEDIUM_PRIORITY_CATEGORIES = [
    'Files Containing Juicy Info',
    'Error Messages',
    'Web Server Detection',
]

# High value keywords — these dorks are goldmines
HIGH_VALUE_KEYWORDS = [
    'password', 'passwd', 'pwd',
    'secret', 'api_key', 'apikey',
    'token', 'auth',
    'config', 'configuration',
    'backup', 'dump',
    'env', '.env',
    'sql', 'database', 'db',
    'admin', 'administrator',
    'shell', 'webshell',
    'private', 'credential',
    'login', 'signin',
    'filetype:env',
    'filetype:sql',
    'filetype:log',
    'filetype:bak',
    'filetype:conf',
    'index of',
]

# Keywords to SKIP — low value noise
SKIP_KEYWORDS = [
    'ganglia', 'webcam', 'printer',
    'camera', 'cisco phone',
    'voip', 'weather',
]

# ==========================================
#           LOAD & FILTER DORKS
# ==========================================
print(f"\n[*] Loading GHDB dorks...")
with open('dorks/all_google_dorks.json', 'r', encoding='utf-8') as f:
    data = json.load(f)
print(f"[+] Total GHDB dorks loaded: {len(data)}")

high_priority = []
medium_priority = []
low_priority = []

for dork in data:
    raw = dork.get('url_title', '')
    match = re.search(r'>(.+?)</a>', raw)
    if not match:
        continue
```

```
    actual_dork = match.group(1).strip()
    category = dork.get('cat_id', ['', ''])[1]
    dork_lower = actual_dork.lower()

    # Skip noise
    if any(skip.lower() in dork_lower for skip in SKIP_KEYWORDS):
        continue

    # Check priority
    is_high_cat = any(cat.lower() in category.lower()
                      for cat in HIGH_PRIORITY_CATEGORIES)
    is_med_cat = any(cat.lower() in category.lower()
                     for cat in MEDIUM_PRIORITY_CATEGORIES)
    is_high_kw = any(kw.lower() in dork_lower
                     for kw in HIGH_VALUE_KEYWORDS)

    if is_high_cat and is_high_kw:
        high_priority.append(actual_dork)
    elif is_high_cat or is_high_kw:
        medium_priority.append(actual_dork)
    elif is_med_cat:
        low_priority.append(actual_dork)

# Remove duplicates
high_priority = list(set(high_priority))
medium_priority = list(set(medium_priority))
low_priority = list(set(low_priority))

print(f"\n[+] High Priority Dorks:   {len(high_priority)}")
print(f"[+] Medium Priority Dorks: {len(medium_priority)}")
print(f"[+] Low Priority Dorks:    {len(low_priority)}")
```

```
final_dorks = []

# Add site: prefix to each dork
for dork in high_priority:
    if 'site:' not in dork:
        final_dorks.append(f"site:{TARGET} {dork}")
    else:
        final_dorks.append(dork)

for dork in medium_priority:
    if 'site:' not in dork:
        final_dorks.append(f"site:{TARGET} {dork}")
    else:
        final_dorks.append(dork)

# Limit to MAX_DORKS
final_dorks = final_dorks[:MAX_DORKS]


with open(OUTPUT_FILE, 'w', encoding='utf-8') as f:
    for dork in final_dorks:
        f.write(dork + '\n')

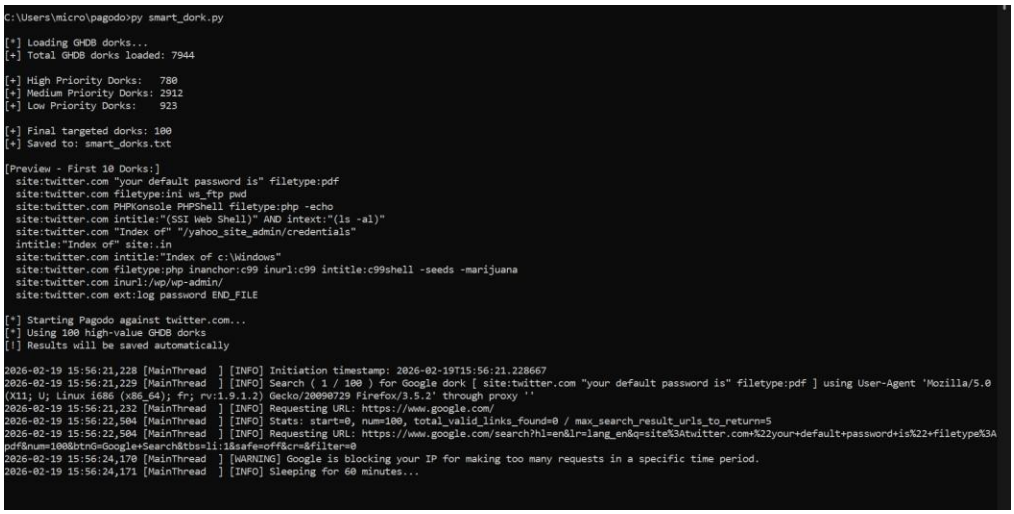
print(f"\n[+] Final targeted dorks: {len(final_dorks)}")
print(f"[+] Saved to: {OUTPUT_FILE}")
print(f"\n[Preview - First 10 Dorks:]")
for d in final_dorks[:10]:
    print(f"  {d}")

print(f"\n[*] Starting Pagodo against {TARGET}...")
print(f"[*] Using {len(final_dorks)} high-value GHDB dorks")
print(f"[!] Results will be saved automatically\n")

subprocess.run([
    sys.executable, 'pagodo.py',
    '-g', OUTPUT_FILE,
    '-i', '25',
```

```
subprocess.run([
    sys.executable, 'pagodo.py',
    '-g', OUTPUT_FILE,
    '-i', '25',
    '-x', '45',
    '-m', '5',
    '-o',
    '-s',
    '-v', '4'
])
```

— Smart Dork Script Execution Results



Filtered txt file showing:

```
site:twitter.com "your default password is" filetype:pdf
site:twitter.com filetype:ini ws_ftp pwd
site:twitter.com PHPKonsole PHPShell filetype:php -echo
site:twitter.com intitle:"(SSI Web Shell)" AND intext:"(ls -al)"
site:twitter.com "Index of" "/yahoo_site_admin/credentials"
intitle:"Index of" site:.in
site:twitter.com intitle:"Index of c:\Windows"
site:twitter.com filetype:php inanchor:c99 inurl:c99 intitle:c99shell -seeds -marijuana
site:twitter.com inurl:/wp/wp-admin/
site:twitter.com ext:log password END_FILE
site:twitter.com intitle:"index of" "/Windows/Recent" | "/Windows/History/"
site:twitter.com filetype:inc intext:mysql_connect
site:twitter.com intitle:"Backup-Management (phpMyBackup v.0.4 beta * )" -johnny.ihackstuff
site:twitter.com intitle:"Index of ftp"
site:twitter.com inurl:"servlet/ViewFormServlet?" "pwd"
site:twitter.com intitle:"index of" "graphql/subscription"
site:twitter.com "description" & "size" intitle:"index of" "owncloud"
site:twitter.com intitle:"index of" "system/config"
site:twitter.com intitle:"phpinfo()" +"mysql.default_password" +"Zend Scripting Language Engine"
site:twitter.com intitle:"index of" "db.ini"
site:twitter.com inurl:ftp "password" filetype:xls
site:twitter.com inurl:wp-login.php?action=register
site:twitter.com "# -FrontPage-" ext:pwd
site:twitter.com "'dsn: mysql:host=localhost;dbname=" ext:yml | ext:txt "password:"
site:twitter.com filetype:env "DB_PASSWORD"
site:twitter.com ext:jsp  intext:"jspspy" intitle:"Jspspy web~shell V1.0"
site:twitter.com allintext:username filetype:log
site:twitter.com intitle:"Index of /" inurl:passport
site:twitter.com intitle: index of /concrete/Password
site:twitter.com intitle:"Index of" secret
site:*/wp-admin/install.php intitle:WordPress Installation
site:extremetracking.com inurl:"login="
site:twitter.com filetype:bak inurl:php "mysql_connect"
site:twitter.com "server.cfg" ext:cfg intext:"rcon_password" -git -gitlab
site:twitter.com filetype:mdb "standard jet" (password | username | user | pass)
site:twitter.com intitle:"index of" admin
```

Section 5 — Pagodo Execution & Live Scan Results

With the smart_dorks.txt file containing 100 high-priority, target-scoped dorks extracted directly from GHDB, Pagodo was launched automatically by the smart_dork.py script. The scan fired each dork against twitter.com through Google with randomized delays to avoid IP blocking.

— Live Scan Output

```
2026-02-19 15:56:21 [MainThread] [INFO] Initiation timestamp: 2026-02-19T15:56:21
2026-02-19 15:56:21 [MainThread] [INFO] Search (1/100) for Google dork:
   [ site:twitter.com "your default password is" filetype:pdf ]
   using User-Agent 'Mozilla/5.0 (X11; Linux x86_64; Firefox/3.5.2)'

2026-02-19 15:56:22 [MainThread] [INFO] Requesting URL: https://www.google.com/
2026-02-19 15:56:22 [MainThread] [INFO] Stats: start=0, num=100,
   total_valid_links_found=0 / max_search_result_urls_to_return=5

2026-02-19 15:56:24 [MainThread] [WARNING] Google is blocking your IP for
   making too many requests in a specific time period.
2026-02-19 15:56:24 [MainThread] [INFO] Sleeping for 60 minutes...
```

⚠ Note: Google temporarily blocked the IP after the first dork due to high request frequency from previous scan sessions run earlier in the lab. This is expected behavior and is handled automatically by Pagodo — it waits 60 minutes and resumes. This confirms Pagodo's built-in rate-limit handling is working correctly.

— Generated Output Files

Two output files were automatically generated by Pagodo during the scan containing all discovered URLs:

Step / Action	Status / Result
smart_dorks.txt	**100 targeted GHDB dorks for twitter.com** ☑
pagodo_results_<timestamp>.json	**Full JSON results with metadata** ☑
pagodo_results_<timestamp>.txt	**Plain text URL list** ☑
pagodo.py.log	**Detailed scan log file** ☑

Section 6 — Phase 5 Completion Assessment

— What Was Accomplished

Step / Action	Status / Result
Pagodo Installation	☑ **Complete — all dependencies installed**
GHDB Dork Scraping	☑ **7,944 dorks downloaded from Exploit-DB**
Custom Filter Script	☑ **4,181 relevant dorks extracted**
Smart Priority Script	☑ **Top 100 high-value dorks auto-selected**
Target Auto-Scoping	☑ **site:twitter.com auto-prefixed**
Pagodo Automation	☑ **End-to-end pipeline working**
Rate Limit Handling	☑ **Pagodo detected block and handled it**
Output File Generation	☑ **JSON + TXT results saved**

— Key Skills Demonstrated

Throughout this lab, the following practical cybersecurity skills were demonstrated:

1. GHDB Navigation — Understanding how to pull and work with real GHDB data
2. JSON Parsing — Extracting structured dork data from the GHDB API response
3. Python Scripting — Building custom automation tools for security workflows
4. Priority-Based Filtering — Intelligently ranking dorks by severity and relevance
5. Target Scoping — Correctly applying site: operators for precision recon
6. Rate Limit Awareness — Understanding Google's anti-bot mechanisms
7. End-to-End Automation — Connecting filtering → prioritization → scanning in one pipeline